ALLIANCE UNIVERSITY · BCA (AI/ML) · DBMS ASSIGNMENT SERIES

03

ASSIGNMENT 3

Database Design Analysis

Online Food Delivery System

SUBMITTED BY
Vimal Kumar Rao

PROGRAMME
BCA (AI/ML)

INSTITUTION
Alliance University

SUBMISSION DATE
24 October 2026

IN THIS ASSIGNMENT

01 Review model

02 Trace dependencies

03 Find anomalies

04 Recommend fixes

A consistent relational design carried forward from Assignments 1 and 2.

1 Objective, Scope and Brief Coverage

This report examines the Online Food Delivery System designed in Assignments 1 and 2: customer accounts and addresses, restaurant menus, delivery partners, orders, order lines and payments. It looks for **redundancy** and **anomalies**, writes down the **functional dependencies**, explains how poor design produces problems, and proposes improvements, most of which are already built into Assignments 5 to 9.

8	6	3NF	5
tables reviewed	redundancy risks found	achieved, with one noted exception	further improvements proposed

	Brief requirement	Where it is covered
✓	**Identify redundancy and insertion / deletion / update anomalies**	Sections 3 and 4 (worked example on real sample data)
✓	**Identify functional dependencies**	Section 5 (every table, plus candidate keys)
✓	**Explain how poor design leads to anomalies**	Section 3 (flat table) and Section 4
✓	**Propose improvements**	Section 7 (applied and proposed, with SQL)

Existing tables and roles

Table	Purpose	Key fields and relationships
Customer	Account and contact details	PK Customer_ID; Email, Phone UNIQUE
Address	Delivery addresses owned by customers	PK Address_ID; FK Customer_ID
Restaurant	Restaurant profile and rating	PK Restaurant_ID
Menu_Item	Restaurant food catalogue	PK Item_ID; FK Restaurant_ID
Delivery_Partner	Delivery worker and vehicle	PK Delivery_ID; Phone, Vehicle_Number UNIQUE
Orders	Order header and life-cycle	PK Order_ID; FKs Customer, Restaurant, Address, Delivery
Order_Item	Items and quantity per order	Composite PK (Order_ID, Item_ID)
Payment	Method, status and date	PK Payment_ID; UNIQUE FK Order_ID

2 From One Flat Table to Eight Tables

The easiest way to see why normalisation matters is to imagine the data stored in a single spreadsheet-style table, which is how many beginners start.

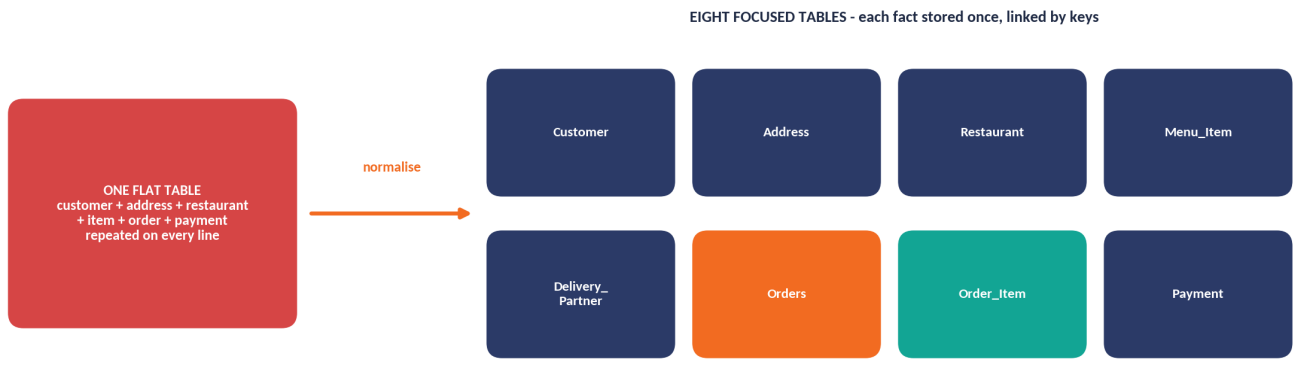


Figure 1. Normalisation splits one repetitive table into eight focused tables linked by keys.

3 How Poor Design Causes Anomalies

Below is the same data for orders 1 to 3 as a **flat table**. Every row repeats the customer, restaurant and item facts.

Order	Customer	Email	Restaurant	Restaurant address	Item	Price	Qty	Payment
1	Asha Menon	asha@example.com	Spice Route Kitchen	Indiranagar, Bengaluru	Chicken Biryani	220	2	Paid
1	Asha Menon	asha@example.com	Spice Route Kitchen	Indiranagar, Bengaluru	Vegetable Meals	150	1	Paid
2	Rahul Nair	rahul@example.com	Green Bowl Cafe	Anna Nagar, Hosur	Veg Buddha Bowl	160	1	Paid
2	Rahul Nair	rahul@example.com	Green Bowl Cafe	Anna Nagar, Hosur	Lemon Iced Tea	70	2	Paid
4	Asha Menon	asha@example.com	Dosa Junction	Railway Station Road, Hosur	Masala Dosa	90	2	Paid
4	Asha Menon	asha@example.com	Dosa Junction	Railway Station Road, Hosur	Filter Coffee	40	2	Paid
6	Meera Iyer	meera@example.com	Spice Route Kitchen	Indiranagar, Bengaluru	Chicken Biryani	220	1	Pending
6	Meera Iyer	meera@example.com	Spice Route Kitchen	Indiranagar, Bengaluru	Paneer Biryani	180	1	Pending

Highlighted rows: order 1 and order 6 are both from Spice Route Kitchen, so its name and address are stored twice already.

Problem	Scenario in this system	Consequence
Redundancy	Asha Menon's name and e-mail repeat on every line of every order she places; the restaurant address repeats for every item sold.	Wasted space, and every copy is a place where the data can disagree.
Update anomaly	Spice Route Kitchen moves. The address must change in every row that mentions it. If one row is missed, the table now holds two addresses for one restaurant.	Conflicting facts; no way to tell which is right.
Insertion anomaly	A new restaurant signs up but has no orders yet. In the flat table a row needs an order to exist, so the restaurant (and its menu) cannot be recorded.	Cannot store a fact until an unrelated fact exists.
Deletion anomaly	Spring Rolls appears on only one order line (order 5). If that order is deleted, the flat table loses the only row that mentions the item, so its name and price vanish with it.	Deleting one fact destroys another.

How the normalised design avoids each one

Restaurant details live once in **Restaurant** (one UPDATE fixes everything). A restaurant and its Menu_Item rows exist independently of any order. Deleting an Order_Item deletes only that line; the menu item remains. Production systems should still keep completed orders rather than hard-delete them.

4 Redundancy and Risk Review of the Earlier Schema

Item	Why it matters	Improvement	Status
Orders.Total_Amount	Can be recalculated from SUM(Quantity x Unit_Price); a stored copy can drift after an edit	Removed; totals computed in queries	Applied
Payment.Amount	Duplicates the order total under a one-payment-per-order rule	Removed; Payment stores method, status, date	Applied
Customer / restaurant details on each order line	Repeats names, e-mails and addresses	Stored once, referenced by key	Applied
Menu price vs Order_Item.Unit_Price	Looks duplicated but records the price the customer agreed to	Kept as a historical snapshot	Applied
Plaintext password field	A leak exposes credentials	Renamed Password_Hash; store salted hashes only	Applied
Address ownership	A simple FK lets an order use another customer's address	Composite FK Orders(Address_ID, Customer_ID)	Applied

5 Functional Dependencies

A functional dependency X → Y means that a value of X determines exactly one value of Y. The left side is the **determinant**.

Relation	Functional dependency	Other determinants / notes
Customer	Customer_ID → Name, Email, Phone, Password_Hash	Email → Customer_ID and Phone → Customer_ID (alternate keys)
Address	Address_ID → Customer_ID, House_No, Street, City, Pincode	Pincode → City (see Section 6)
Restaurant	Restaurant_ID → Restaurant_Name, Phone, Address, Rating	Phone is an alternate key
Menu_Item	Item_ID → Restaurant_ID, Item_Name, Description, Price, Availability	
Delivery_Partner	Delivery_ID → Name, Phone, Vehicle_Number, Status	Phone, Vehicle_Number are alternate keys
Orders	Order_ID → Customer_ID, Restaurant_ID, Address_ID, Delivery_ID, Order_Date, Order_Status	
Order_Item	(Order_ID, Item_ID) → Quantity, Unit_Price	Needs the whole key; neither Order_ID nor Item_ID alone is enough
Payment	Payment_ID → Order_ID, Method, Status, Date	Order_ID → Payment_ID (UNIQUE, 0..1 payment)

6 Normalisation Assessment

Normal form	Rule	How this design meets it
1NF	Atomic values, no repeating groups	Several items on an order are separate Order_Item rows, not a comma-separated list
2NF	No partial dependency on part of a composite key	Quantity and Unit_Price depend on the whole (Order_ID, Item_ID)
3NF	No dependency of a non-key attribute on another non-key attribute	Customer, restaurant, item and partner facts live in their own tables
BCNF	Every determinant is a candidate key	Holds, except Pincode → City in Address (below)

An honest exception: Address

In Address, **Pincode → City** holds in practice, and Address_ID → Pincode, so City depends on the key only *transitively*. Strictly this is a 3NF/BCNF violation. It was kept on purpose: postal-code lookup tables are usually overkill for a delivery app, and one pincode can span localities with different names. If strict normalisation is required, split it as shown in Section 7, item P4.

7 Improvements: Applied and Proposed

#	Change	Status
A1	Remove stored totals	Applied (A5)
A2	Password_Hash instead of Password	Applied (A5)
A3	Composite FK for address ownership	Applied (A5)
A4	UNIQUE, CHECK, DEFAULT rules; indexes on date and status	Applied (A5, A6)
P1	Enforce that every order line belongs to the order's restaurant (composite keys)	Proposed
P2	Block assignment of Offline delivery partners (trigger)	Proposed
P3	Soft delete / retention for orders and payments	Proposed

#	Change	Status
P4	Separate Pincode table to reach strict 3NF	Optional
P5	Fees, tax and discounts modelled and snapshotted per order	Production scope

P1: a rule the schema cannot yet enforce

Today nothing stops an order for restaurant 1 from containing an item of restaurant 2. Assignment 9 (Q15) audits for this and finds no violations in the sample data, but a composite foreign key would make the database refuse it:

```
ALTER TABLE Orders    ADD UNIQUE KEY uq_order_restaurant (Order_ID, Restaurant_ID);
ALTER TABLE Menu_Item ADD UNIQUE KEY uq_item_restaurant  (Item_ID,  Restaurant_ID);
ALTER TABLE Order_Item
  ADD COLUMN Restaurant_ID INT NOT NULL,           -- back-fill before adding the keys
  ADD CONSTRAINT fk_oi_order_rest FOREIGN KEY (Order_ID, Restaurant_ID)
      REFERENCES Orders (Order_ID, Restaurant_ID),
  ADD CONSTRAINT fk_oi_item_rest  FOREIGN KEY (Item_ID, Restaurant_ID)
      REFERENCES Menu_Item (Item_ID, Restaurant_ID);
```

P4: optional strict 3NF for addresses

```
CREATE TABLE Pincode (Pincode VARCHAR(10) PRIMARY KEY, City VARCHAR(100) NOT NULL);
-- Address then keeps only Pincode (FK to Pincode) and drops its own City column.
```

Why P1 to P5 were not applied

The coursework schema is kept identical across Assignments 5 to 9 so the documents stay consistent. These changes are listed as the next design iteration.

8 Conclusion

The initial model already had a sound normalised foundation. The main improvements were to avoid duplicated derived totals, protect credentials, enforce address ownership and formalise value ranges. The review also found two further gaps (restaurant consistency of order lines, and a pincode-to-city dependency) and documents how to close them. These decisions carry forward into Assignments 5 to 9.